

3. Control de versiones y gestión de proyectos

Fundamentos de Git: commits, ramas, merges, tags

Conceptos clave:

Elemento	Descripción	Aplicación en redes
Commit	Registro de cambios en el código	Guardar modificaciones en scripts de configuración
Rama (branch)	Línea paralela de desarrollo	Separar versiones de scripts para distintos dispositivos
Merge	Unión de ramas	Integrar mejoras o correcciones al proyecto principal
Tag	Punto de referencia en el historial	Marcar versiones estables de scripts o APIs

Actividad práctica:

Tarea: Investigar cómo se crear un repositorio en GitHub para un proyecto de automatización de red.

Flujo de trabajo colaborativo: Git Flow, pull requests

Modelos de flujo:

Modelo	Características	Uso recomendado
Git Flow	Separación clara entre desarrollo, pruebas y producción	Proyectos con múltiples colaboradores
Trunk-based development	Desarrollo directo en la rama principal con validaciones	Proyectos pequeños o ágiles
Fork + Pull Request	Copia del repositorio para contribuciones externas	Colaboración abierta o institucional

Buenas prácticas:

- Crear ramas por funcionalidad (*feature/config_vlan*)
- Usar mensajes de commit descriptivos
- Revisar código antes de hacer merge
- Documentar cambios en el README o changelog

Actividad práctica:

Tarea: Simular un flujo Git Flow en equipo para desarrollar un script de monitoreo.

Integración con CI/CD y automatización de despliegues

Conceptos clave:

Elemento	Descripción	Herramientas comunes
CI (Integración continua)	Automatiza pruebas y validaciones al integrar código	GitHub Actions, Jenkins
CD (Despliegue continuo)	Automatiza la entrega del software en producción	Ansible, Docker, Terraform
Pipeline	Flujo automatizado de pasos (build, test, deploy)	YAML para definir procesos

Aplicación en redes:

- Validar scripts antes de ejecutarlos en dispositivos reales
- Desplegar configuraciones en múltiples routers/switches
- Monitorear cambios y revertir si hay fallos

Actividad práctica:

Tarea: Investigar cómo crear un pipeline básico en GitHub Actions que valide un script YAML antes de desplegarlo.

Auditoría de cambios y documentación técnica

Elementos clave:

Elemento	Función	Herramientas
Historial de cambios	Registro de quién, cuándo y qué se modificó	Git log, changelog.md
Documentación técnica	Explicación del código, decisiones y uso	README.md, Wiki, Google Docs
Licencias y contribuciones	Reglas de uso y colaboración	LICENSE, CONTRIBUTING.md

Buenas prácticas:

- Documentar cada módulo o script con propósito y parámetros
- Usar plantillas para documentación estandarizada
- Mantener changelog actualizado por versión
- Incluir ejemplos de uso y pruebas

Actividad práctica:

Tarea: Redactar la documentación técnica de un script de configuración de red.